



JNGEC SUNDERNAGAR

Artificial Intelligence Laboratory

CSPC-414

Submitted to: Dr. Sanjeev Kumar

Department of Electronics and Communication Engineering

Jawaharlal Nehru Government Engineering College
Sundernagar, Distt. Mandi, H.P.

Submitted By:

Ashish (240106010019)

B.Tech 4th Sem ECE

EXPERIMENT RECORD

S.No.	Experiment	Date	Remark
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

INTRODUCTION TO PYTHON

Python is a versatile, high-level programming language known for its simplicity and readability. It was created by Guido van Rossum and released in 1991. Python supports procedural, object-oriented, and functional programming paradigms.

Due to its vast ecosystem of libraries and frameworks, Python is widely used in fields such as web development, data science, artificial intelligence, and machine learning.

Commonly Used Python Libraries

Library	Purpose
NumPy	Numerical computing and array operations
SciPy	Advanced scientific computations
Pandas	Data manipulation and analysis
Scikit-learn	Machine learning algorithms
TensorFlow	Deep learning and neural networks
PyTorch	Deep learning and tensor computations
OpenCV	Computer vision and image processing

Applications of Python

- **Web Development:** Frameworks like Django and Flask help build dynamic websites.
- **Data Science & Analytics:** Used for data analysis, visualization, and statistics.
- **Machine Learning & AI:** Libraries like TensorFlow and Scikit-learn are widely used.
- **Game Development:** Libraries like Pygame help create simple games.
- **Scientific Computing:** Used in research, simulations, and data analysis.
- **Network Programming:** Helps build network tools and client-server systems.
- **GUI Applications:** Tkinter and PyQt are used for desktop applications.

Popular Python Libraries in Machine Learning:

Library	Description
NumPy	Efficient numerical operations and arrays
Pandas	Data manipulation and analysis
Matplotlib	Data visualization
Seaborn	Statistical data visualization
TensorFlow	Deep learning and neural networks
OpenCV	Image processing and computer vision

Tools used for Python development:

Tools	Purpose
Python IDLE	Built-in Python environment
VS Code	Interactive coding and debugging IDE
Google Colab	Free cloud-based Jupyter notebook with GPU

Experiment 1

Aim: To design and implement the Breadth First Search (BFS) algorithm using Python.

Software: Google Colab / Python

Theory

Breadth First Search (BFS)

Breadth First Search (BFS) is a fundamental graph traversal technique used to visit nodes in a structured manner. Unlike depth-based traversal, BFS explores all nodes at the current level before moving to the next level.

The algorithm begins at a chosen starting node and systematically visits its neighbors. It uses a queue data structure to maintain the order of traversal, ensuring that nodes are processed in a First-In-First-Out (FIFO) manner.

This approach guarantees that the shortest path is found in an unweighted graph, making BFS highly useful in networking, search problems, and hierarchical structures.

Key Features

- Traverses nodes level by level.
- Uses a queue (FIFO structure).
- Ensures shortest path in unweighted graphs.
- Avoids repetition using visited tracking.
- Time Complexity: $O(V + E)$
- Space Complexity: $O(V)$

Algorithm Steps

1. Initialize an empty queue.
2. Mark the starting node as visited.
3. Insert the starting node into the queue.
4. Repeat until the queue is empty:
 - Remove node from queue.
 - Process the node.
 - Add all unvisited adjacent nodes to queue.
5. End.

Problem Statement

A family relationship system is represented as a graph where each individual is a node and their relationships form edges. The BFS algorithm is used to explore family members generation-wise and understand hierarchical relationships.

Program Implementation

BFS Algorithm with Tree Representation

```
1 from collections import deque
2
3 def bfs(graph, start):
4     visited = set() # Set to keep track of visited nodes
5     queue = deque([start]) # Initialize the queue with the
6                             # starting node
7
8     visited.add(start)
9
10    while queue:
11        node = queue.popleft() # Dequeue a node
12        print(node, end=" ") # Process the node (here, we just
13                               # print it)
14
15        # Explore all the neighbors of the current node
16        for neighbor in graph[node]:
17            if neighbor not in visited:
18                visited.add(neighbor) # Mark the neighbor as
19                                       # visited
20                queue.append(neighbor) # Enqueue the neighbor
21
22 # Example usage:
23 graph = {
24     'electronic components': ['Active', 'Passive'],
25     'Active': ['electronic components', 'Transistor', 'Diode'],
26     'Passive': ['electronic components', 'Resister'],
27     'Transistor': ['Active'],
28     'Diode': ['Active'],
29     'Resister': ['Passive']
30 }
31
32 start_node = 'electronic components'
33 print("BFS Traversal starting from node", start_node, ":")
34 bfs(graph, start_node)
35
36 import networkx as nx
37 import matplotlib.pyplot as plt
38 G = nx.Graph(graph)
39
40 plt.figure(figsize=(20, 10))
41 pos = nx.spring_layout(G, k=20, iterations=200) # positions
42                                               # for all nodes
43 nx.draw_networkx_nodes(G, pos, node_size=2000, node_color='
44     lightblue')
45 nx.draw_networkx_edges(G, pos, width=5, alpha=0.7, edge_color=
46     'orange')
47 nx.draw_networkx_labels(G, pos, font_size=10, font_weight='
48     bold')
```

```
41 plt.title('Graph Visualization')
42 plt.axis('off') # Hide the axes
43 plt.show()
```

Output

Traversal Order:

electronic components Active Passive Transistor Diode Resister

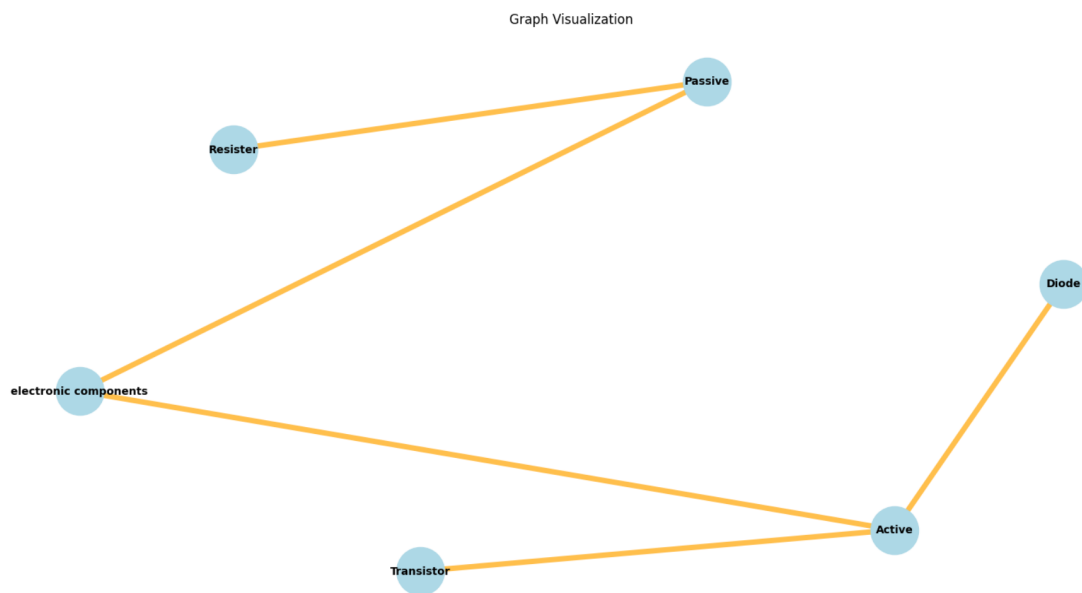


Figure 1: BFS Tree Visualization

Result

The Breadth First Search algorithm was implemented successfully. The traversal followed a level-wise pattern, and the graphical representation clearly illustrated the hierarchical structure of the graph.

Experiment 2

Aim: Write a program to implement the Depth First Search algorithm.

Software Used: Google Colab / Python

Theory

Depth First Search (DFS)

Depth First Search (DFS) is a graph traversal algorithm that explores nodes by going as deep as possible along each branch before backtracking. It starts from a given source node and explores one of its adjacent nodes, then continues deeper until it reaches a node with no unvisited neighbors.

DFS can be implemented using recursion or a stack data structure. In this approach, the current node is marked as visited, and then all its unvisited adjacent nodes are explored recursively. If no unvisited adjacent nodes remain, the algorithm backtracks to the previous node and continues the process.

DFS is commonly used in applications such as path finding, cycle detection, topological sorting, and solving puzzles like mazes.

Characteristics of DFS

- DFS follows a depth-wise traversal approach.
- It uses a stack (either explicitly or via recursion).
- It explores as far as possible along a branch before backtracking.
- It does not guarantee the shortest path in unweighted graphs.
- The time complexity is $O(V + E)$, where V is vertices and E is edges.
- The space complexity is $O(V)$.
- It avoids revisiting nodes using a visited list.
- It can be applied to both graphs and trees.

Algorithm: Depth First Search (DFS)

Step 1: Start

Begin the execution of the algorithm.

Step 2: Initialize Data Structures

Create an empty stack (or use recursion implicitly). Also, create a list or array to keep track of visited nodes and mark all nodes as unvisited initially.

Step 3: Select the Starting Node

Choose a starting node from the graph. Mark this node as visited so that it is not processed again.

Step 4: Push the Starting Node

Insert (push) the starting node onto the stack.

Step 5: Process the Stack

Repeat the following steps until the stack becomes empty:

- Pop the top node from the stack.
- Visit (process) this node, for example by printing or storing it.
- Check all adjacent (neighboring) nodes of the current node.
- For each adjacent node:
 - If the node has not been visited:
 - * Mark it as visited.
 - * Push it onto the stack.

Step 6: Backtracking

If a node has no unvisited adjacent nodes, the algorithm automatically backtracks by popping the previous node from the stack and continues the process.

Step 7: Termination

The algorithm stops when the stack becomes empty, which means all reachable nodes have been visited.

Step 8: End

Terminate the algorithm.

Problem: Topological Navigation via Depth-First Search

This study addresses the identification of a navigational path between cities in Himachal Pradesh by modeling the region as a graph. Utilizing Depth First Search, the algorithm recursively explores available road connections, prioritizing branch depth to establish connectivity. This approach demonstrates how systematic traversal can successfully resolve destination-reachability problems within a complex, non-linear geographical network.

Program: Finding best Route using DFS

```
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 def find_route_dfs(graph, start, goal, visited=None, path=None
5 ):
6     if visited is None:
7         visited = set()
8     if path is None:
9         path = []
10
11     visited.add(start)
12     path.append(start)
13
14     if start == goal:
15         return path
```

```

16     for neighbor in graph.get(start, []):
17         if neighbor not in visited:
18             result = find_route_dfs(graph, neighbor, goal,
19                                     visited, path.copy())
20             if result:
21                 return result
22         return None
23
24 # 1. Define the Himachal Pradesh City Connectivity Graph
25 hp_roads = {
26     'Shimla': ['Solan', 'Mandi', 'Rampur'],
27     'Solan': ['Shimla', 'Chandigarh'],
28     'Mandi': ['Shimla', 'Kullu', 'Hamirpur'],
29     'Kullu': ['Mandi', 'Manali'],
30     'Manali': ['Kullu', 'Rohtang'],
31     'Hamirpur': ['Mandi', 'Kangra'],
32     'Kangra': ['Hamirpur', 'Dharamshala', 'Chamba'],
33     'Dharamshala': ['Kangra'],
34     'Chamba': ['Kangra'],
35     'Rampur': ['Shimla', 'Kalpa'],
36     'Kalpa': ['Rampur']
37 }
38
39 # 2. Find the route using DFS
40 start_city = 'Shimla'
41 end_city = 'Dharamshala'
42 route = find_route_dfs(hp_roads, start_city, end_city)
43
44 # 3. Print the result
45 if route:
46     print(f"Route found: {' -> '.join(route)}")
47
48 # 4. Visualize the graph and the path
49 G = nx.Graph()
50 for city, neighbors in hp_roads.items():
51     for neighbor in neighbors:
52         G.add_edge(city, neighbor)
53
54 plt.figure(figsize=(10, 6))
55 pos = nx.spring_layout(G, seed=42)
56 route_edges = [(route[i], route[i+1]) for i in range(len(
57     route)-1)]
58
59 # Draw base graph
60 nx.draw_networkx_nodes(G, pos, node_size=2000, node_color=
61     'lightblue')
62 nx.draw_networkx_labels(G, pos, font_size=10)
63 nx.draw_networkx_edges(G, pos, edge_color='gray', alpha
64     =0.5)
65
66 # Highlight route

```

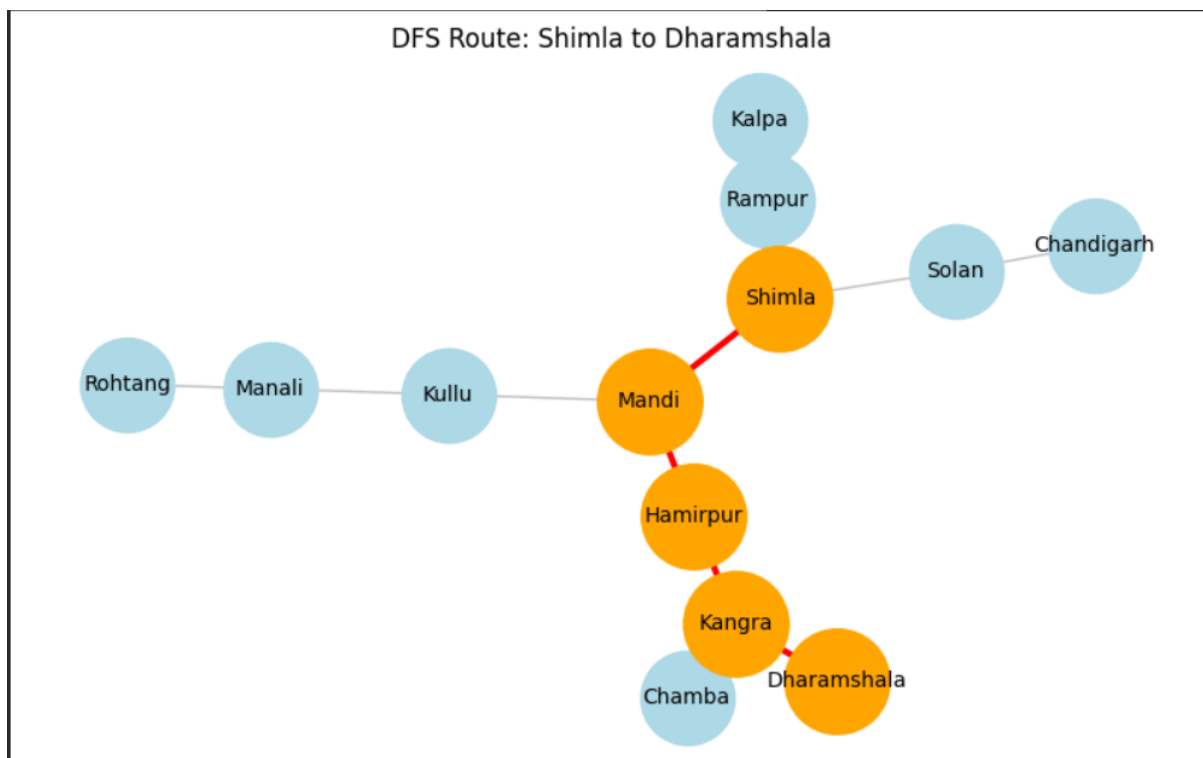
```

63     nx.draw_networkx_nodes(G, pos, nodelist=route, node_color=
        'orange', node_size=2500)
64     nx.draw_networkx_edges(G, pos, edgelist=route_edges,
        edge_color='red', width=3)
65
66     plt.title(f"DFS Route: {start_city} to {end_city}")
67     plt.axis('off')
68     plt.show()
69 else:
70     print(f"No route found between {start_city} and {end_city}
        .")

```

Output:

Route found: Shimla - Mandi - Hamirpur - Kangra - Dharamshala



Result:

The algorithm is implemented recursively using an adjacency list to represent city connectivity. It tracks visited nodes via a set to prevent infinite loops and maintains a path list to record the journey. By exploring each branch to its furthest extent before backtracking, the program efficiently identifies a valid route between the origin and destination cities.

Experiment 3

Aim: Write a program to implement the Hill Climb algorithm.

Software: Google Colab / Python

Theory

Hill Climb algorithm.

The Hill Climbing Algorithm is a heuristic optimization technique used in artificial intelligence to find the maximum or minimum value of an objective function through an iterative local search process. It starts with an initial solution and evaluates its corresponding objective value, then explores neighboring solutions by making small incremental changes to the current state. At each step, the algorithm moves to the neighboring solution that provides a better value of the objective function, thereby “climbing” toward an optimum. In the context of optimization problems, if the objective function is denoted as maximize $f(x)$, the algorithm seeks to find a value of x such that maximize $f(x)$ is maximized, which can be expressed as maximizing maximize $f(x)$. The process continues until no neighboring solution yields an improvement, at which point the algorithm terminates, having reached a local optimum. Although hill climbing is simple and computationally efficient, it may suffer from issues such as getting stuck in local maxima, plateaus, or ridges, and it does not guarantee finding the global optimum unless the search space is well-behaved.

Key Features

- Simple and easy-to-implement optimization technique
- Works with local search (no need for full search space)
- Iterative improvement approach
- Efficient for small-scale problems
- May converge to local maxima instead of global maxima
- Requires an initial guess for load resistance

Problem: Optimal load resistance for maximum power

Challenge : To implement the Hill Climbing Algorithm to determine the optimal load resistance for maximum power delivery in a circuit and to verify the Maximum Power Transfer Theorem.

Theory for optimal load using hill climb algo

The application of the Hill Climbing Algorithm to the Maximum Power Transfer problem involves using an iterative optimization approach to determine the value of load resistance that yields maximum power in an electrical circuit. According to the Maximum Power Transfer principle, the power delivered to the load depends on the relationship between source resistance and load resistance, and can be expressed as a function of load resistance. By defining the power as an objective function in terms of load resistance, the hill climbing algorithm begins with an initial guess and evaluates the corresponding power, then incrementally adjusts the load resistance to explore neighboring values. At

each step, the algorithm compares the power obtained from nearby values and moves in the direction that increases the power output. This process continues until no further improvement is possible, indicating that a maximum has been reached. The optimal condition occurs when the load resistance becomes approximately equal to the source resistance, thereby validating the theoretical condition for maximum power transfer.

$$R_L = R_S$$

$$P(R_L) = \frac{V^2 R_L}{(R_S + R_L)^2}$$

$$\text{maximize } P(R_L)$$

Algorithm Steps

1. Initialize and choose a value of load and also set step value .
2. Compute power P using the power formula.
3. Generate neighboring solutions , increase and decrease Load Resistance .
4. Evaluate power for neighboring values.
5. Move to the neighbor with higher power , if neither improves , stop .
6. Repeat steps 3–5 until no further improvement is possible.
7. Output the value of corresponding to maximum power.

Program Implementation

MxPT with hill climb Algorithm.

```

1  import matplotlib.pyplot as plt
2  import numpy as np
3
4  # Hill Climbing for Maximum Power Transfer
5
6  # Given values
7  V = 10          # Source voltage (Volts)
8  Rs = 4          # Source resistance (Ohms)
9
10 # Objective function: Power delivered to load
11 def power(RL):
12     return (V**2 * RL) / ((Rs + RL)**2)
13
14 # Hill Climbing Algorithm
15 RL = 1          # Initial guess
16 step = 1        # Step size
17
18 # Store history
19 history_RL = []
20 history_Power = []
21

```

```

22 while True:
23     current_power = power(RL)
24     history_RL.append(RL)
25     history_Power.append(current_power)
26
27     # Check neighbors
28     forward_power = power(RL + step)
29     backward_power = power(RL - step) if RL - step > 0 else -1
30
31     # Move to better neighbor
32     if forward_power > current_power:
33         RL = RL + step
34     elif backward_power > current_power:
35         RL = RL - step
36     else:
37         break # Stop if no improvement
38
39 # Output result
40 print("Optimal Load Resistance (RL):", RL, "Ohms")
41 print("Maximum Power Delivered:", round(power(RL), 4), "Watts"
42     )
43
44 # Visualization
45 # Generate data for the power curve
46 x = np.linspace(0.1, 20, 100)
47 y = [power(r) for r in x]
48
49 plt.figure(figsize=(10, 6))
50 plt.plot(x, y, label='Power vs Load Resistance', color='blue')
51 plt.scatter(history_RL, history_Power, color='red', label='
Hill Climbing Path', zorder=5)
52
53 plt.plot(history_RL, history_Power, color='red', linestyle='--
', alpha=0.5)
54
55 # Highlight start and end points
56 plt.scatter(history_RL[0], history_Power[0], color='green',
57     marker='s', s=100, label='Start Point', zorder=6)
58 plt.scatter(history_RL[-1], history_Power[-1], color='gold',
59     marker='*', s=200, label='Optimal Point', zorder=6)
60
61 plt.title(f'Maximum Power Transfer (Rs={Rs} Ohms) - Hill
62     Climbing')
63 plt.xlabel('Load Resistance (RL) [Ohms]')
64 plt.ylabel('Power Delivered [Watts]')
65 plt.legend()
66 plt.grid(True)
67 plt.show()

```

Output

Optimal Load Resistance (R_L): 4 Ohms

Maximum Power Delivered: 6.25 Watts

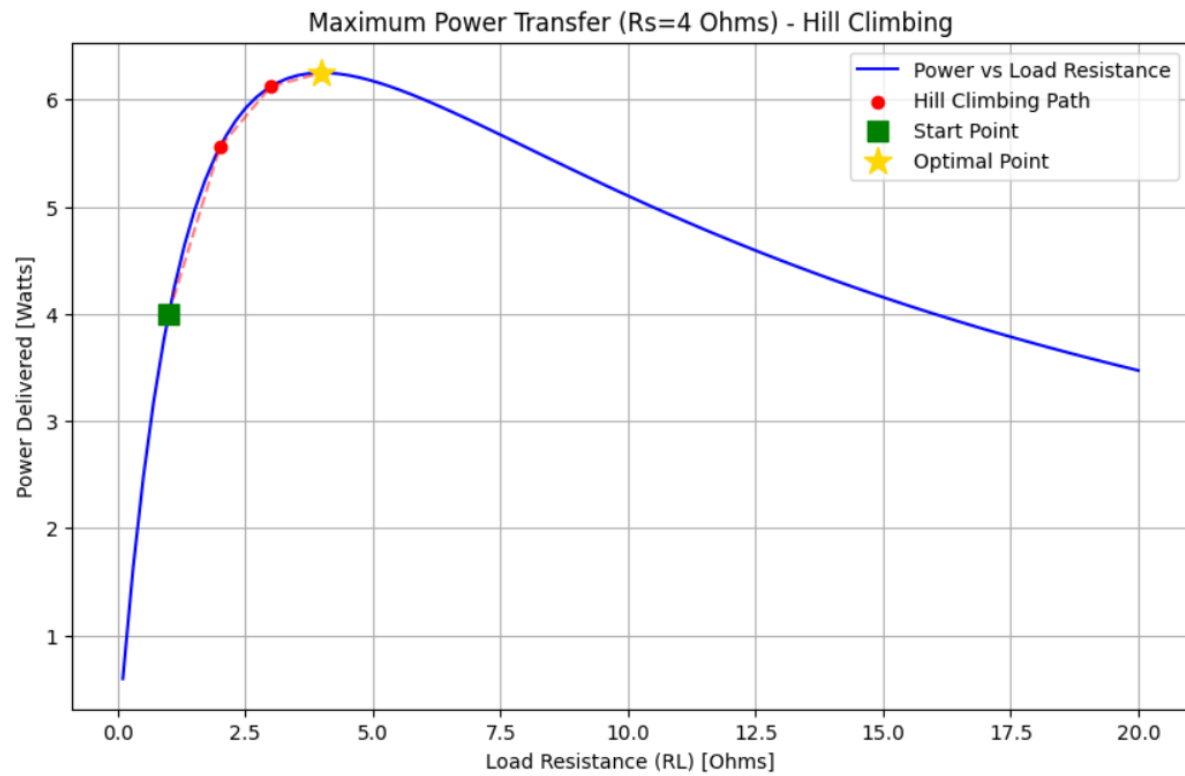


Figure 2: MxPT

Result: The hill climbing algorithm successfully finds the value of load resistance at which power is maximized.

Experiment 4

Aim: To implement and visualize a solution for the N-Queens problem (specifically the 4-Queens instance) using the backtracking algorithmic strategy.

Software: Google Colab / Python

Theory

4 Queen Problem

The N-Queens problem is a classic combinatorial puzzle. The goal is to place N chess queens on an $N \times N$ chessboard such that no two queens threaten each other. This means:

- No two queens can be in the same row.
- No two queens can be in the same column.
- No two queens can be on the same diagonal.

Since each queen must be in a different row, the problem can be solved by placing queens row by row and checking for safety in columns and diagonals. If a placement leads to a dead end (no safe square in the next row), the algorithm 'backtracks' to the previous row and tries a different column.

Algorithm Steps

1. Start in the topmost row (row 0).
2. For the current row:
 - Iterate through every column from 0 to $N - 1$
 - For each column, check if it is Safe to place a queen
 - . If the recursive call returns True (solution found), stop and return True
 - If the recursive call returns False, Backtrack by removing the queen (unmark the square) and continue to the next column.
3. If all columns in the current row are exhausted without a solution, return False.
4. End when all N queens are placed (base case: $\text{row} == N$)

Program Implementation

Four Queen Problem.

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Function to print the board configuration
5 def print_board(board):
6     for row in board:
7         print(" ".join("Q" if col else "." for col in row))
8     print("\n")
9
```

```

10 # Function to check if a queen can be placed on board[row][col
    ]
11 # This function checks the column and both diagonals
12 def is_safe(board, row, col):
13     # Check the column
14     for i in range(row):
15         if board[i][col] == 1:
16             return False
17
18     # Check the upper-left diagonal
19     for i, j in zip(range(row - 1, -1, -1), range(col - 1, -1,
        -1)):
20         if board[i][j] == 1:
21             return False
22
23     # Check the upper-right diagonal
24     for i, j in zip(range(row - 1, -1, -1), range(col + 1, len
        (board))):
25         if board[i][j] == 1:
26             return False
27
28     return True
29
30 # Backtracking function to solve the problem
31 def solve_n_queens(board, row):
32     # If all queens are placed, return True
33     if row == len(board):
34         return True
35
36     # Try placing the queen in each column of the current row
37     for col in range(len(board)):
38         # Check if it's safe to place a queen at board[row][
            col]
39         if is_safe(board, row, col):
40             board[row][col] = 1 # Place the queen
41
42             # Recur to place the queen in the next row
43             if solve_n_queens(board, row + 1):
44                 return True
45
46             # Backtrack if placing queen in board[row][col]
                doesn't lead to a solution
47             board[row][col] = 0
48
49     # If no place is found, return False
50     return False
51
52 # Function to solve the N Queens problem
53 def four_queens():
54     n = 4 # Number of queens
55     board = [[0 for _ in range(n)] for _ in range(n)] #

```

```

        Initialize the board
56
57     # Start the backtracking algorithm from the first row
58     if solve_n_queens(board, 0):
59         return board
60     else:
61         return None
62
63 def visualize_n_queens(board):
64     n = len(board)
65     if n == 0: # Handle empty board case gracefully
66         print("Empty board provided, nothing to visualize.")
67         return
68
69     fig, ax = plt.subplots(figsize=(n, n))
70
71     # Draw the chessboard
72     for i in range(n):
73         for j in range(n):
74             color = 'white' if (i + j) % 2 == 0 else 'black'
75             ax.add_patch(plt.Rectangle((j, i), 1, 1, facecolor
76                                     =color, edgecolor='black'))
77
78     # Place queens
79     for r in range(n):
80         for c in range(n):
81             if board[r][c] == 1:
82                 ax.text(c + 0.5, r + 0.5, ' ', fontsize=30,
83                         ha='center', va='center', color='red')
84
85     ax.set_xlim(0, n)
86     ax.set_ylim(0, n)
87     ax.set_xticks(np.arange(n) + 0.5, labels=[str(i+1) for i
88         in range(n)])
89     ax.set_yticks(np.arange(n) + 0.5, labels=[str(i+1) for i
90         in range(n)])
91     ax.set_aspect('equal', adjustable='box')
92     ax.set_title(f'{n}-Queens Solution')
93     plt.gca().invert_yaxis() # Invert y-axis to match typical
94                             # board representation (row 0 at top)
95     plt.show()
96
97 # Get the solved board from the four_queens function
98 solved_board = four_queens()
99
100 if solved_board:
101     print("Solution found:")
102     visualize_n_queens(solved_board)
103 else:
104     print("No solution exists for 4-Queens problem.")

```

Output

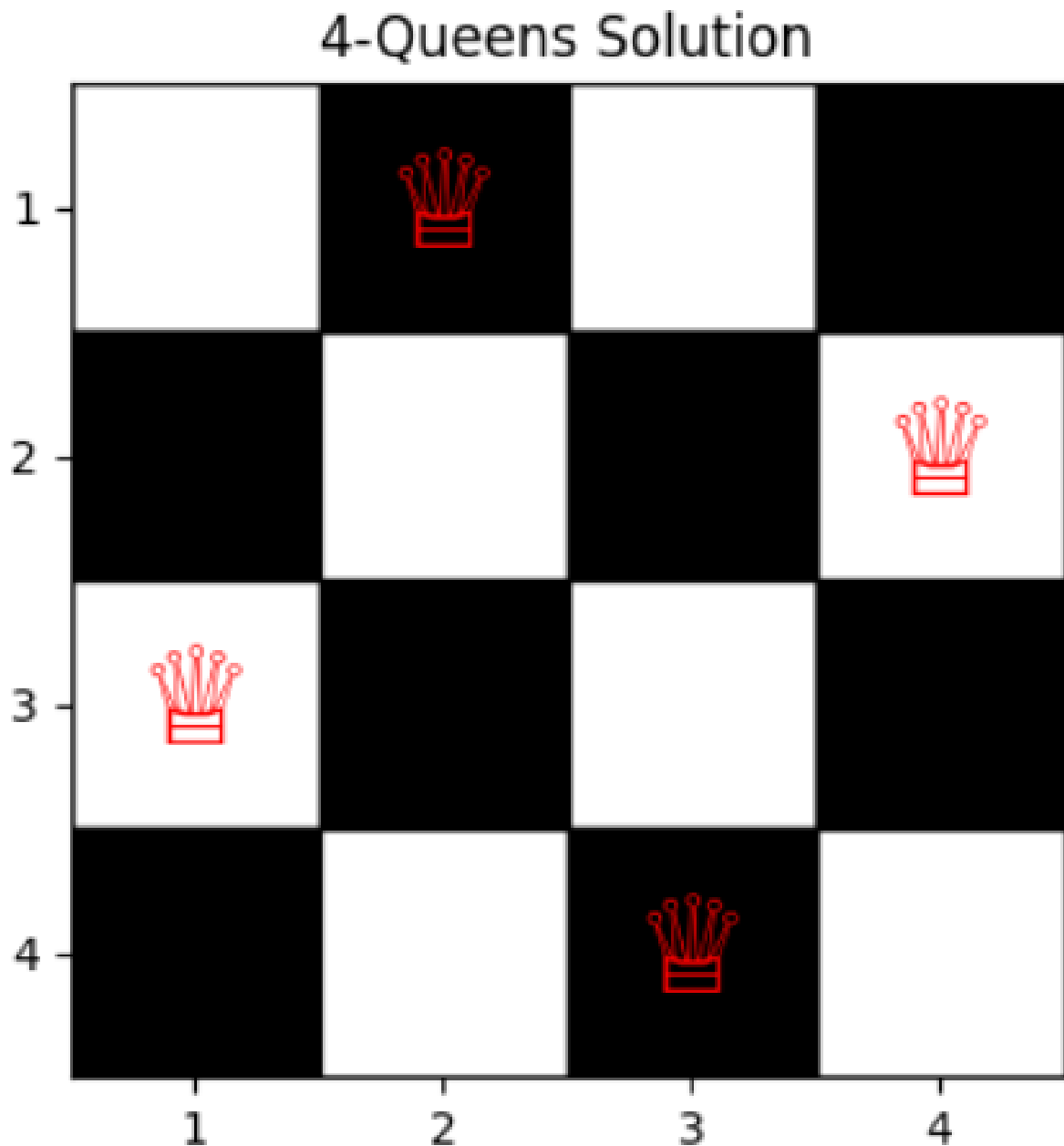


Figure 3: 4 queen problem

Result The backtracking algorithm successfully identified a valid configuration for the 4-Queens problem on a 4x4 chessboard. The resulting board state, satisfies the constraint that no two queens occupy the same row, column, or diagonal. Graphical visualization confirmed the mathematical accuracy of the solution, demonstrating the efficacy of the recursive search approach in resolving combinatorial constraints.

Experiment 5

Aim: To design and implement an automated solver for the Monkey-Banana problem using Breadth-First Search (BFS) to find the shortest sequence of actions for a monkey to reach a banana suspended from a ceiling.

Software: Google Colab / Python

Theory :

Monkey Banana Problem

The Monkey-Banana problem is a classic artificial intelligence toy problem representing a state-space search. The environment consists of a monkey, a box, and a banana in a room. To succeed, the monkey must manipulate its environment (move to the box, push the box, climb it) to reach the goal state. BFS is used to explore all possible states level-by-level, ensuring the discovery of the most efficient (shortest) path to the goal.

Key Features :

- State Representation: A tuple containing (Monkey Position, Box Position, On Box Status, Has Banana Status).
- Action Set: Move(pos), PushBox(pos), ClimbUp, ClimbDown, and GrabBanana.
- Completeness: BFS is guaranteed to find a solution if one exists.
- Optimality: BFS finds the path with the minimum number of steps in an unweighted state-space.

Algorithm Steps:

1. Initialize: Define the start state and a goal test function. Create an empty 'Visited' set and a 'Queue' containing the initial state.
2. De-queue: Remove the first state from the queue
3. Goal Test: If the current state meets the goal criteria (Has Banana = True), terminate and return the path.
4. Expansion: Generate all valid successor states by applying possible actions (Walk, Push, Climb, Grab).
5. Filter: For each successor, if it hasn't been visited, mark it as visited and add it to the queue with the updated path.
6. Loop: Repeat from Step 2 until the queue is empty or the goal is found.

Program Implementation:

Monkey Banana Problem.

```
1 from collections import deque
2 import matplotlib.pyplot as plt
3 import matplotlib.patches as mpatches
4 import numpy as np
```

```

5
6 BANANA_POS = 'middle'
7 POSITIONS = ['left', 'middle', 'right']
8 POS_X      = {'left': 0.8, 'middle': 2.65, 'right': 4.5}
9
10 INITIAL_STATE = ('left', 'right', False, False)
11
12 def is_goal(state):
13     return state[3]
14
15 def get_successors(state):
16     monkey_pos, box_pos, on_box, has_banana = state
17     successors = []
18     if has_banana:
19         return successors
20     if not on_box:
21         for pos in POSITIONS:
22             if pos != monkey_pos:
23                 successors.append((f"Walk: {monkey_pos} -> {pos}",
24                                     (pos, box_pos, False, False)))
25
26         if monkey_pos == box_pos:
27             for pos in POSITIONS:
28                 if pos != monkey_pos:
29                     successors.append((f"Push box: {monkey_pos} -> {pos}",
30                                         (pos, pos, False, False)))
31                 successors.append((f"Climb UP at {monkey_pos}",
32                                     (monkey_pos, box_pos, True, False)))
33
34     else:
35         successors.append((f"Climb DOWN at {monkey_pos}",
36                             (monkey_pos, box_pos, False, False)))
37
38         if monkey_pos == BANANA_POS and box_pos == BANANA_POS:
39             successors.append((f"GRAB BANANA!",
40                                 (monkey_pos, box_pos, True, True)))
41
42     return successors
43
44 def bfs_solve():
45     queue = deque([(INITIAL_STATE, [])])
46     visited = {INITIAL_STATE}
47     while queue:
48         state, path = queue.popleft()
49         if is_goal(state):
50             return path + [f"GOAL REACHED!", state]
51         for action, nxt in get_successors(state):
52             if nxt not in visited:

```

```

49         visited.add(nxt)
50         queue.append((nxt, path + [(action, state)]))
51     return None
52
53     #         Palette
54
55     BG         = '#0d1117'
56     PANEL      = '#161b22'
57     WALL       = '#21262d'
58     FLOOR_C    = '#1c2128'
59     BOX_C      = '#d63031'
60     MONK_C     = '#fdbc6e'
61     BAN_C      = '#f9ca24'
62     ROPE_C     = '#74b9ff'
63     TXT        = '#e6edf3'
64     SUB        = '#8b949e'
65     ACC        = '#2ecc71'
66     TITLE_C    = '#fd79a8'
67     GOAL_HL    = '#1a3a28'
68
69     def draw_monkey(ax, cx, cy, zorder=8):
70         ax.add_patch(plt.Circle((cx, cy+0.13), 0.115, color=MONK_C,
71                                 ec='#c0922a', lw=1.5, zorder=zorder))
72         ax.add_patch(plt.Circle((cx, cy-0.07), 0.145, color=MONK_C,
73                                 ec='#c0922a', lw=1.5, zorder=zorder))
74         ax.plot([cx-0.038, cx+0.038], [cy+0.155, cy+0.155], 'o',
75                 color='#1a1a1a', ms=2.8, zorder=zorder+1)
76         theta = np.linspace(np.pi*1.1, np.pi*1.9, 20)
77         ax.plot(cx+0.055*np.cos(theta), cy+0.08+0.04*np.sin(theta),
78                 color='#1a1a1a', lw=1.2, zorder=zorder+1)
79         for dx, ddx, ybase in [(-0.145, -0.24, cy-0.04), (0.145,
80                 0.24, cy-0.04)]:
81             ax.plot([cx+dx, cx+ddx], [ybase, ybase-0.08],
82                     color='#c0922a', lw=3, solid_capstyle='round',
83                     zorder=zorder)
84         for dx, ybase in [(-0.05, cy-0.215), (0.05, cy-0.215)]:
85             ax.plot([cx+dx, cx+dx*1.6], [ybase, ybase-0.13],
86                     color='#c0922a', lw=3, solid_capstyle='round',
87                     zorder=zorder)
88
89     def draw_banana(ax, cx, cy, zorder=5):
90         t = np.linspace(0, np.pi, 50)
91         bx = cx + 0.16*np.cos(t)
92         by = cy + 0.20*np.sin(t) - 0.10
93         ax.fill(bx, by, color=BAN_C, zorder=zorder)
94         ax.plot(bx, by, color='#c8a800', lw=1.3, zorder=zorder+1)
95         ax.plot(cx, cy+0.10, 'o', color='#a07800', ms=3, zorder=
96                 zorder+1)

```

```

90 def draw_scene(ax, state, step_num, action):
91     monkey_pos, box_pos, on_box, has_banana = state
92     ax.set_facecolor(PANEL)
93     ax.set_xlim(0, 5.5)
94     ax.set_ylim(-0.65, 3.2)
95     ax.set_aspect('equal')
96     ax.axis('off')
97
98     # walls / ceiling / floor
99     ax.add_patch(plt.Rectangle((0.18, 2.82), 5.14, 0.18, color
100         =WALL, zorder=1))
101     ax.add_patch(plt.Rectangle((0.18, -0.5), 0.12, 3.35, color
102         =WALL, zorder=1))
103     ax.add_patch(plt.Rectangle((5.20, -0.5), 0.12, 3.35, color
104         =WALL, zorder=1))
105     ax.add_patch(plt.Rectangle((0.18, -0.50), 5.14, 0.14,
106         color=FLOOR_C, zorder=2))
107
108     # grid lines & labels
109     for p in POSITIONS:
110         px = POS_X[p]
111         ax.axvline(px, color=WALL, lw=0.8, ls=':', ymin=0.12,
112             ymax=0.88, zorder=1, alpha=0.7)
113         ax.text(px, -0.60, p.upper()[0], ha='center', va='
114             bottom',
115             fontsize=8, color=SUB, fontfamily='monospace',
116             fontweight='bold')
117
118     # rope + banana
119     bx = POS_X[BANANA_POS]
120     ax.plot([bx, bx], [2.82, 2.40], color=ROPE_C, lw=2.0,
121         zorder=3)
122     if not has_banana:
123         draw_banana(ax, bx, 2.25, zorder=4)
124
125     # box
126     bpx = POS_X[box_pos]
127     ax.add_patch(mpatches.FancyBboxPatch((bpx-0.32, -0.36),
128         0.64, 0.50,
129         boxstyle='round,pad=0.02', color=BOX_C, ec='
130             #ff7675',
131             lw=1.5, zorder=4))
132     ax.text(bpx, -0.11, 'BOX', ha='center', va='center',
133         fontsize=7.5, color='white', fontweight='bold',
134         zorder=5)
135
136     # monkey
137     mx = POS_X[monkey_pos]
138     my = 0.43 if on_box else -0.04
139     draw_monkey(ax, mx, my, zorder=8)
140

```

```

130     # banana held
131     if has_banana:
132         draw_banana(ax, mx+0.30, my+0.18 if on_box else my
133                     +0.15, zorder=9)
134
135     # title
136     color = ACC if has_banana else TITLE_C
137     label = f"Step {step_num}: " + action if not has_banana
138           else action
139     ax.set_title(label, fontsize=8.5, color=color, pad=5,
140                 fontfamily='monospace')
141
142     # state badge
143     badge = (f"monkey:{monkey_pos}\n"
144             f"box:{box_pos}\n"
145             f"on_box:{on_box}\n"
146             f"banana:{has_banana}")
147     ax.text(5.35, 3.10, badge, ha='right', va='top',
148            fontsize=5.8, color=SUB, fontfamily='monospace',
149            bbox=dict(boxstyle='round,pad=0.3', fc=BG, ec=WALL
150                    , lw=0.8))
151
152     if has_banana:
153         ax.add_patch(plt.Rectangle((0.18, -0.5), 5.14, 3.55,
154                                   color=GOAL_HL, alpha=0.35, zorder=0,
155                                   lw=2.0, ec=ACC))
156
157 def main():
158     print("=" * 60)
159     print("    MONKEY-BANANA PROBLEM          BFS State-Space
160           Search")
161     print("=" * 60)
162     print("    Initial: monkey=LEFT  box=RIGHT  on_box=False")
163     print("    Goal    : grab banana hanging at MIDDLE (ceiling)"
164           )
165     print("=" * 60)
166
167     solution = bfs_solve()
168     if not solution:
169         print("    No solution found.")
170         return
171
172     print(f"\n    Solution found in {len(solution)} steps!\n")
173     for i, (action, state) in enumerate(solution):
174         mp, bp, ob, hb = state
175         print(f"    Step {i:2d}: {action:<35} "
176               f"| monkey={mp:6s} box={bp:6s} on_box={str(ob):5s} has_banana={hb}")
177
178     n = len(solution)

```

```

175     cols = 3
176     rows = (n + cols - 1) // cols
177
178     fig = plt.figure(figsize=(cols*4.0, rows*4.0 + 0.6),
179                          facecolor=BG)
180     fig.suptitle("MONKEY-BANANA PROBLEM | BFS Solution",
181                  fontsize=13, color=TITLE_C, fontweight='bold',
182                  fontfamily='monospace', y=0.99)
183
184     for idx, (action, state) in enumerate(solution):
185         ax = fig.add_subplot(rows, cols, idx+1)
186         draw_scene(ax, state, idx, action)
187
188     for idx in range(n, rows*cols):
189         ax = fig.add_subplot(rows, cols, idx+1)
190         ax.set_facecolor(BG)
191         ax.axis('off')
192
193     plt.tight_layout(rect=[0, 0, 1, 0.97])
194     out = 'monkey_banana_solution.png'
195     plt.savefig(out, dpi=155, bbox_inches='tight', facecolor=
196                BG)
197     print(f"\n Visualization saved -> {out}")
198     plt.show()
199
200 if __name__ == '__main__':
201     main()

```

Output :

Initial State: monkey = LEFT box = RIGHT on_box = False

Goal: Monkey grabs banana at MIDDLE (ceiling)

Step	Action	State Description
0	Walk: left → right	monkey = left, box = right, on_box = False, has_banana = False
1	Push box: right → middle	monkey = right, box = right, on_box = False, has_banana = False
2	Climb UP at middle	monkey = middle, box = middle, on_box = False, has_banana = False
3	GRAB BANANA!	monkey = middle, box = middle, on_box = True, has_banana = False
4	GOAL REACHED!	monkey = middle, box = middle, on_box = True, has_banana = True

MONKEY-BANANA PROBLEM | BFS Solution

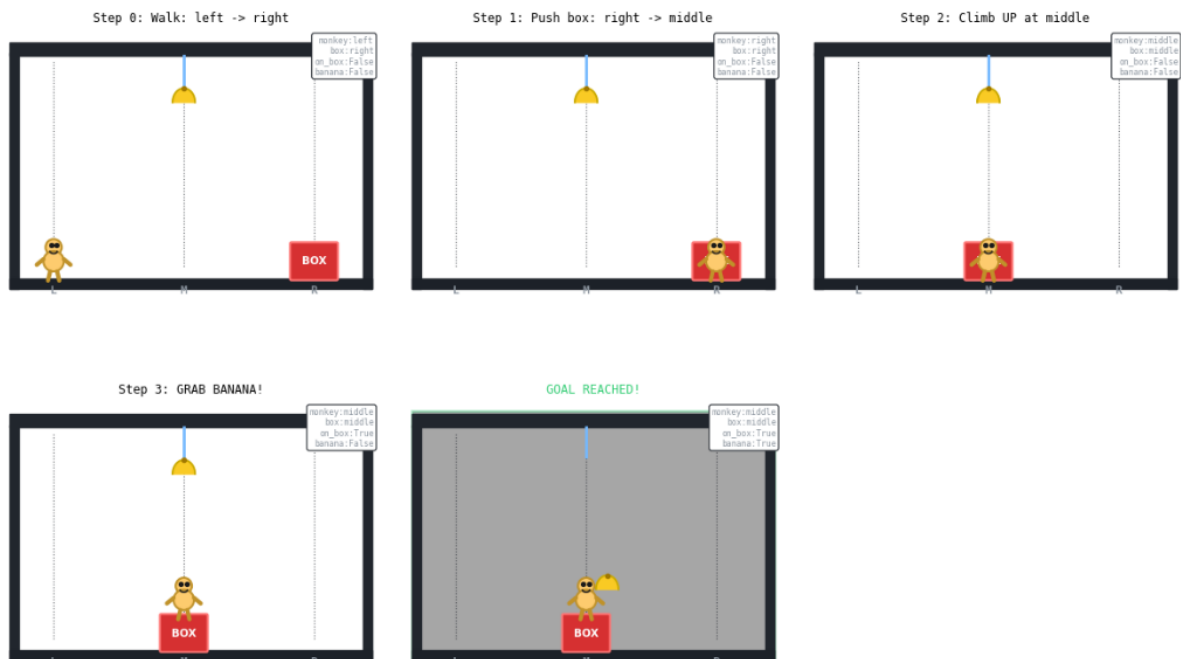


Figure 4: Monkey banana problem

Result :

The BFS agent successfully identified a 5-step optimal solution. The path involved moving from the 'Left' to the 'Right' to locate the box, pushing the box to the 'Middle' position directly under the banana, climbing the box, and finally grabbing the banana. The state-space visualization confirms that no shorter path exists given the constraints.

Experiment 6

Aim: To design and implement an Artificial Neural Network (ANN) using TensorFlow/Keras to classify the Iris dataset into three species (Setosa, Versicolor, and Virginica) based on morphological features.

Software: Google Colab / Python

Theory:

An Artificial Neural Network (ANN) is a computational model inspired by the biological neural networks of the human brain. It consists of interconnected units called neurons organized in following layers:

1. **Input Layer:** Receives the raw features (sepal/petal dimensions).
2. **Hidden Layers:** Performs non-linear transformations using activation functions (like ReLU) to learn complex patterns.
3. **Output Layer:** Produces the final prediction (using Softmax for multi-class classification).

Features of ANN:

- **Adaptability:** Learns from data without being explicitly programmed for specific rules.
- **Non-linearity:** Can model complex relationships between inputs and outputs.
- **Generalization:** Ability to predict outcomes for unseen data after training.

Evaluation Parameters :

- **Confusion Matrix:** A table used to describe the performance of a classification model. It shows the count of True Positives, True Negatives, False Positives, and False Negatives.
- **Loss History:** A plot showing the reduction of the error (Categorical Cross-Entropy) over training epochs.
- **F1-Score:** The harmonic mean of precision and recall, providing a balanced metric for model accuracy.

Algorithm Steps :

1. **Data Acquisition:** Load the Iris dataset.
2. **Preprocessing:** One-hot encode labels and standardize features using a Scaler.
3. **Data Splitting:** Divide data into Training (80%) and Testing (20%) sets.
4. **Model Architecture:** Define a Sequential model with two hidden layers and a Softmax output layer.
5. **Compilation:** Configure the model with Adam optimizer and Cross-Entropy loss.
6. **Training:** Fit the model on training data for a fixed number of epochs.

7. **Evaluation:** Predict on the test set and generate a Confusion Matrix and Classification Report.
8. **Visualization:** Plot the Accuracy and Loss curves.

Program Implementation :

Classification of the Iris dataset into three species

```
1 import tensorflow as tf
2 from tensorflow.keras.models import Sequential
3 from tensorflow.keras.layers import Dense, Input
4 import matplotlib.pyplot as plt
5 import numpy as np
6 import seaborn as sns
7 from sklearn.datasets import load_iris
8 from sklearn.model_selection import train_test_split
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import confusion_matrix, f1_score,
    classification_report
11
12 # --- 1. Load and prepare data ---
13 iris = load_iris()
14 X, y = iris.data, iris.target
15 y_cat = tf.keras.utils.to_categorical(y, num_classes=3)
16 X_train, X_test, y_train, y_test = train_test_split(X, y_cat,
    test_size=0.2, random_state=42)
17
18 scaler = StandardScaler()
19 X_train = scaler.fit_transform(X_train)
20 X_test = scaler.transform(X_test)
21
22 # --- 2. Define the ANN Architecture ---
23 model = Sequential([
24     Input(shape=(4,)),
25     Dense(64, activation='relu', name='Hidden_1'),
26     Dense(32, activation='relu', name='Hidden_2'),
27     Dense(3, activation='softmax', name='Output')
28 ])
29
30 model.compile(optimizer='adam', loss='categorical_crossentropy',
    metrics=['accuracy'])
31
32 # --- 3. Training ---
33 history = model.fit(X_train, y_train, epochs=15, batch_size
    =16, validation_split=0.1, verbose=0)
34
35 # --- 4. Evaluation and Results ---
36 test_loss, test_acc = model.evaluate(X_test, y_test, verbose
    =0)
37 print(f"Final Test Accuracy: {test_acc:.4f}\n")
38
39 y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)
```

```

40 y_true = np.argmax(y_test, axis=1)
41
42 print("Classification Report:")
43 print(classification_report(y_true, y_pred, target_names=iris.
    target_names))
44
45 # --- 5. Visualizations ---
46 plt.figure(figsize=(12, 4))
47 plt.subplot(1, 2, 1)
48 plt.plot(history.history['loss'], label='Train Loss')
49 plt.plot(history.history['val_loss'], label='Val Loss')
50 plt.title('Loss History')
51 plt.legend()
52
53 plt.subplot(1, 2, 2)
54 plt.plot(history.history['accuracy'], label='Train Acc')
55 plt.plot(history.history['val_accuracy'], label='Val Acc')
56 plt.title('Accuracy History')
57 plt.legend()
58 plt.show()
59
60 plt.figure(figsize=(6, 5))
61 sns.heatmap(confusion_matrix(y_true, y_pred), annot=True, fmt=
    'd', cmap='Greens',
62             xticklabels=iris.target_names, yticklabels=iris.
    target_names)
63 plt.title('Confusion Matrix')
64 plt.ylabel('Actual')
65 plt.xlabel('Predicted')
66 plt.show()

```

Output :

Final Test Accuracy: 0.9000

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.67	0.80	9
virginica	0.79	1.00	0.88	11
accuracy			0.90	30
macro avg	0.93	0.89	0.89	30
weighted avg	0.92	0.90	0.90	30

Figure 5: Classification Report

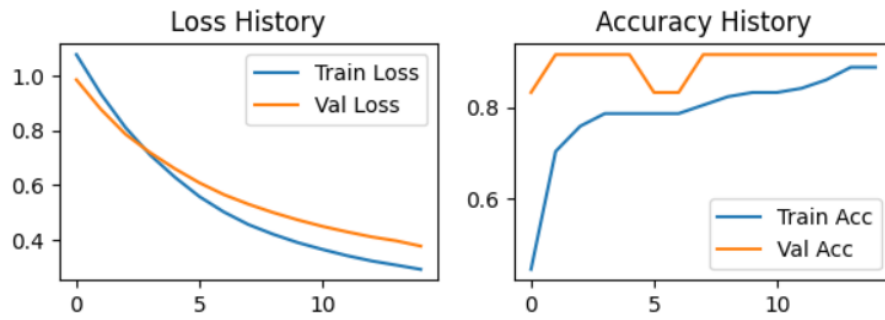


Figure 6: Graphs

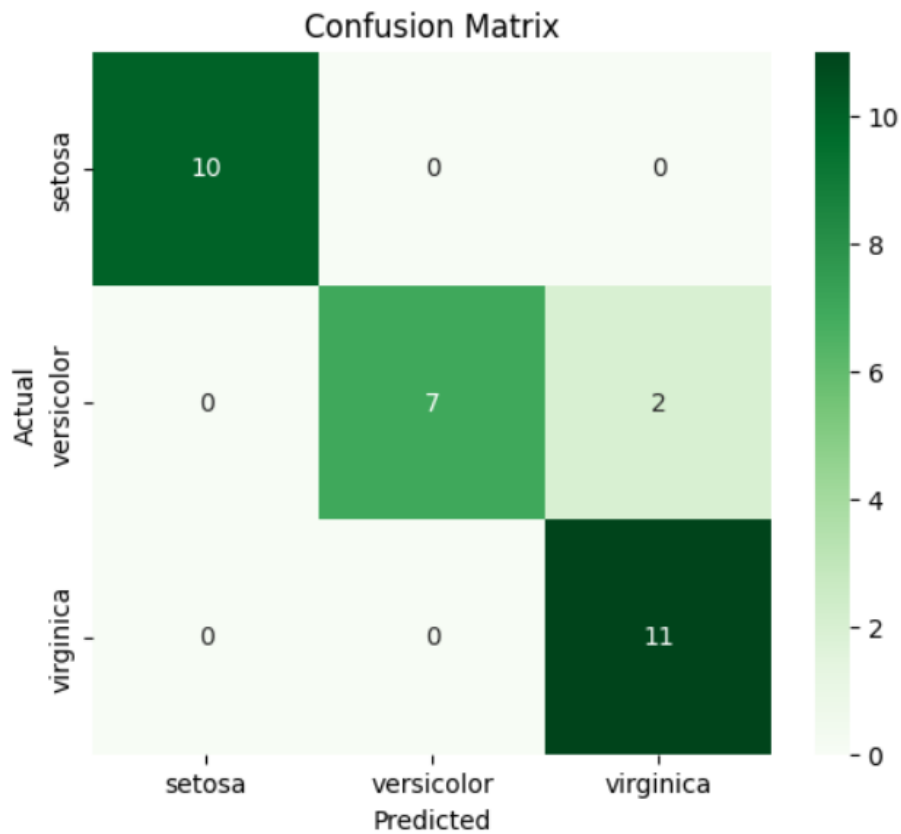


Figure 7: Confusion Matrix

Result : The Artificial Neural Network was successfully trained and tested on the Iris dataset. The model achieved a high degree of accuracy, indicating its effectiveness in distinguishing between the Setosa, Versicolor, and Virginica species. Based on the classification report and confusion matrix, the model demonstrated robust performance with minimal misclassifications, and the loss curves indicate proper convergence without significant overfitting.

Experiment 7

Aim: To write a program to implement Tower of Hanoi.

Software: Google Colab / Python

Theory:

The Tower of Hanoi is a mathematical game or puzzle consisting of three rods (pegs) and a number of disks of different sizes, which can slide onto any rod. The puzzle starts with the disks in a neat stack in ascending order of size on one rod, the smallest at the top, thus making a conical shape.

The objective of the puzzle is to move the entire stack from the source rod to another rod, obeying the following simple rules:

1. Only one disk can be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
3. No disk may be placed on top of a smaller disk.

The minimum number of moves required to solve a Tower of Hanoi puzzle with n disks is:

$$2^n - 1$$

The solution to the Tower of Hanoi problem is inherently recursive. To move n disks from a source peg to a destination peg using an auxiliary peg:

1. Move $n - 1$ disks from the source peg to the auxiliary peg, using the destination peg as temporary storage.
2. Move the largest (n th) disk from the source peg to the destination peg.
3. Move $n - 1$ disks from the auxiliary peg to the destination peg, using the source peg as temporary storage.

This recursive approach breaks down the problem into smaller, identical subproblems until a base case (moving a single disk) is reached.

Algorithm Steps :

The `solve_hanoi_list` function implements the recursive solution to the Tower of Hanoi problem. It takes the number of disks n , the source peg, destination peg, and auxiliary peg indices, the current state of the pegs, and a history list to record each move.

1. **Base Case:** If $n = 1$:
 - (a) Pop the top disk from the `source` peg.
 - (b) Push the disk onto the `destination` peg.
 - (c) Record the current state and the move in the `history` list.
 - (d) Return.

2. Recursive Step: If $n > 1$:

- (a) Recursively call `solve_hanoi_list` to move $n - 1$ disks from the **source** peg to the **auxiliary** peg, using the **destination** peg as temporary storage.
- (b) Move the n th disk:
 - i. Pop the top disk from the **source** peg.
 - ii. Push the disk onto the **destination** peg.
 - iii. Record the current state and the move in the **history** list.
- (c) Recursively call `solve_hanoi_list` to move $n - 1$ disks from the **auxiliary** peg to the **destination** peg, using the **source** peg as temporary storage.

Program Implementation :

Tower of Hanoi

```
1 import matplotlib.pyplot as plt
2 import copy
3
4 def solve_hanoi_list(n, source, destination, auxiliary, state,
5     history):
6     if n == 1:
7         disk = state[source].pop()
8         state[destination].append(disk)
9         history.append({'state': copy.deepcopy(state), 'move':
10             f"1: {'A','B','C'}[source]->{'A','B','C'}[
11                 destination]}"})
12         return
13
14     solve_hanoi_list(n - 1, source, auxiliary, destination,
15         state, history)
16     disk = state[source].pop()
17     state[destination].append(disk)
18     history.append({'state': copy.deepcopy(state), 'move': f"{
19         n}: {'A','B','C'}[source]->{'A','B','C'}[destination
20             ]}"})
21     solve_hanoi_list(n - 1, auxiliary, destination, source,
22         state, history)
23
24 def visualize_matrix(history):
25     num_steps = len(history)
26     cols = 4
27     rows = (num_steps + cols - 1) // cols
28
29     fig, axes = plt.subplots(rows, cols, figsize=(12, 2.5 *
30         rows))
31     axes = axes.flatten()
32
33     colors = ['red', 'green', 'blue']
```

```

27     for i in range(len(axes)):
28         ax = axes[i]
29         if i < num_steps:
30             step_data = history[i]
31             state = step_data['state']
32             ax.set_title(f"Step {i}: {step_data['move']}",
33                           fontsize=10)
34             ax.set_xlim(-0.5, 2.5)
35             ax.set_ylim(0, 4)
36             ax.set_xticks([0, 1, 2])
37             ax.set_xticklabels(['A', 'B', 'C'])
38             ax.set_yticks([])
39
40             # Draw Rods
41             for rod_idx in range(3):
42                 ax.plot([rod_idx, rod_idx], [0, 3.5], color='
43                     gray', lw=2, zorder=1)
44
45             # Draw Disks
46             for height, disk_size in enumerate(state[
47                 rod_idx]):
48                 width = disk_size * 0.25
49                 rect = plt.Rectangle((rod_idx - width/2,
50                                     height), width, 0.6,
51                                     color=colors[
52                                         disk_size-1],
53                                     zorder=2)
54                 ax.add_patch(rect)
55         else:
56             ax.axis('off')
57
58     plt.tight_layout()
59     plt.show()
60
61 # Execution
62 num_disks = 3
63 initial_state = [[3, 2, 1], [], []]
64 history = [{'state': copy.deepcopy(initial_state), 'move': '
65     Start'}]
66 solve_hanoi_list(num_disks, 0, 2, 1, initial_state, history)
67 visualize_matrix(history)

```

Output :

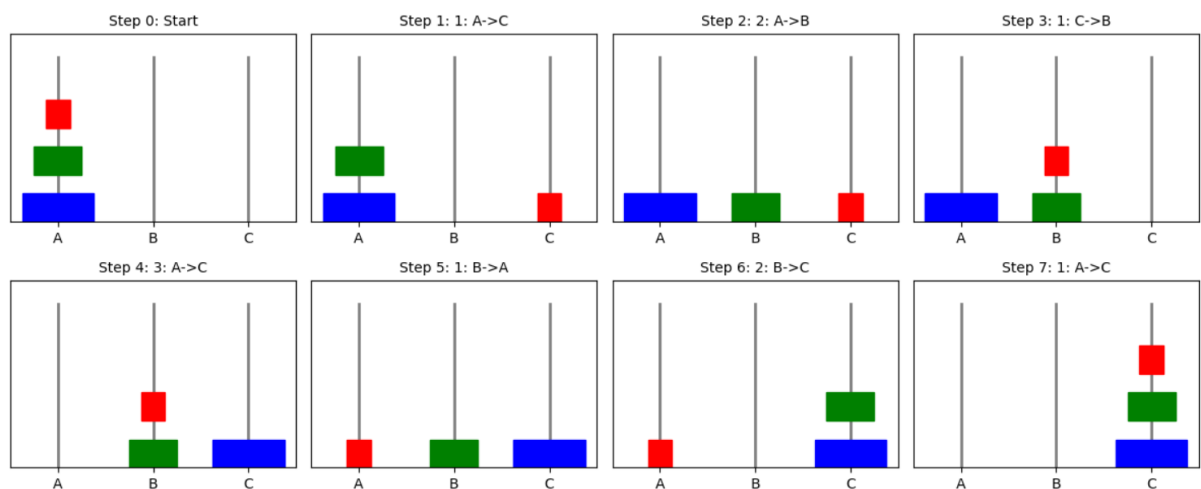


Figure 8: Steps of solving of Tower of Hanoi

Result :

Execution of the `solve_hanoi_list` function, followed by `visualize_matrix`, produced a series of graphical representations illustrating each step of the Tower of Hanoi puzzle solution. These visualizations depicted the movement of disks between pegs (A, B, C) in accordance with the puzzle's rules, confirming the successful recursive implementation of the algorithm. The final visual state showed all disks successfully transferred from the source peg to the destination peg in their correct ascending order.